TECHNICAL BRIEFING

x402 over Tari Ootle

Pay-per-request HTTP payments on Tari's confidential contract layer — how the x402 protocol, the Sapient wallet, and a new settlement rail fit together, and what was built and verified.

PotR3288 · October 31, 2026

Live-verified on Tari esmeralda testnet (happy path + refund, commits e82cf1c…aee37c8)

Companion brief for the Sapient developer: sapient-developer-brief.pdf

1. What x402 is

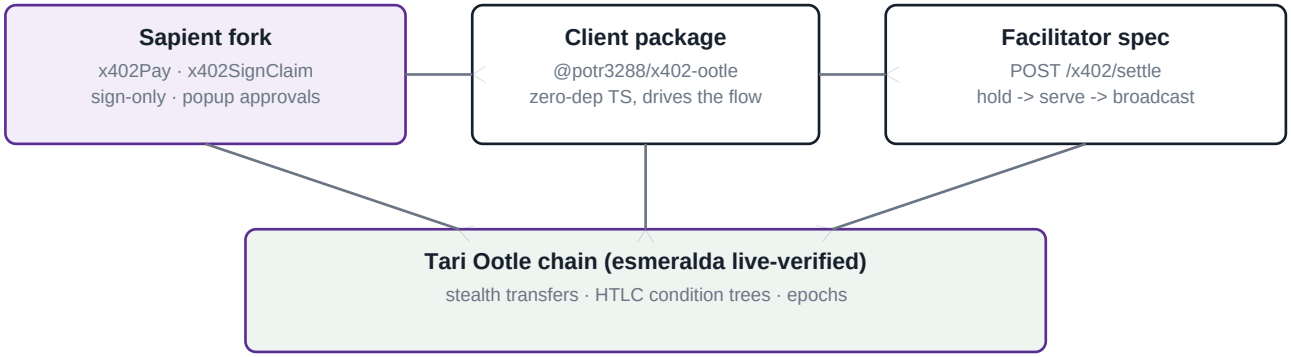
x402 is an open protocol for internet-native payments, built on the long-unused HTTP 402 *Payment Required* status code. When a client requests a protected resource, the server answers 402 with a machine-readable quote describing what the resource costs and how to pay. The client's wallet pays, presents proof, and the server releases the resource. No accounts, no API keys, no subscription plumbing — payment is just part of HTTP. The original x402 rails settle on EVM chains with stablecoins; the protocol is chain-agnostic by design, which is what this project exploits.

Two roles matter beyond the client and server. A **facilitator** is infrastructure that validates and settles payments on the payer's behalf, and a **scheme** defines how a particular chain's payment is constructed and verified. Our work defines an Ootle scheme and makes the Ootle indexer itself the facilitator.

2. Tari Ootle and Sapient

Tari Ootle is Tari's digital-assets layer: a confidential-contract chain where value lives in **stealth outputs** — amounts and ownership encrypted — and spending is gated by **condition trees**, Merkle-structured scripts whose leaves can require hash preimages, epoch windows, or possession of specific keys. Two primitives do the heavy lifting here: a stealth transfer can be locked behind a two-leaf tree (hash-lock OR time-locked refund), and an epoch is the chain's native clock.

Sapient is the Tari Ootle browser wallet (a Chrome extension by chironbuilds, testnet-stage). It injects `window.tari` into pages, keeps keys on-device, renders a human approval popup for every transaction request, and speaks natively to stealth transfers and HTLC condition trees. It is the only surface with all three properties the payment rail needs — browser integration, on-device keys, and native HTLCs — which is why the fork builds on it.



3. What was built

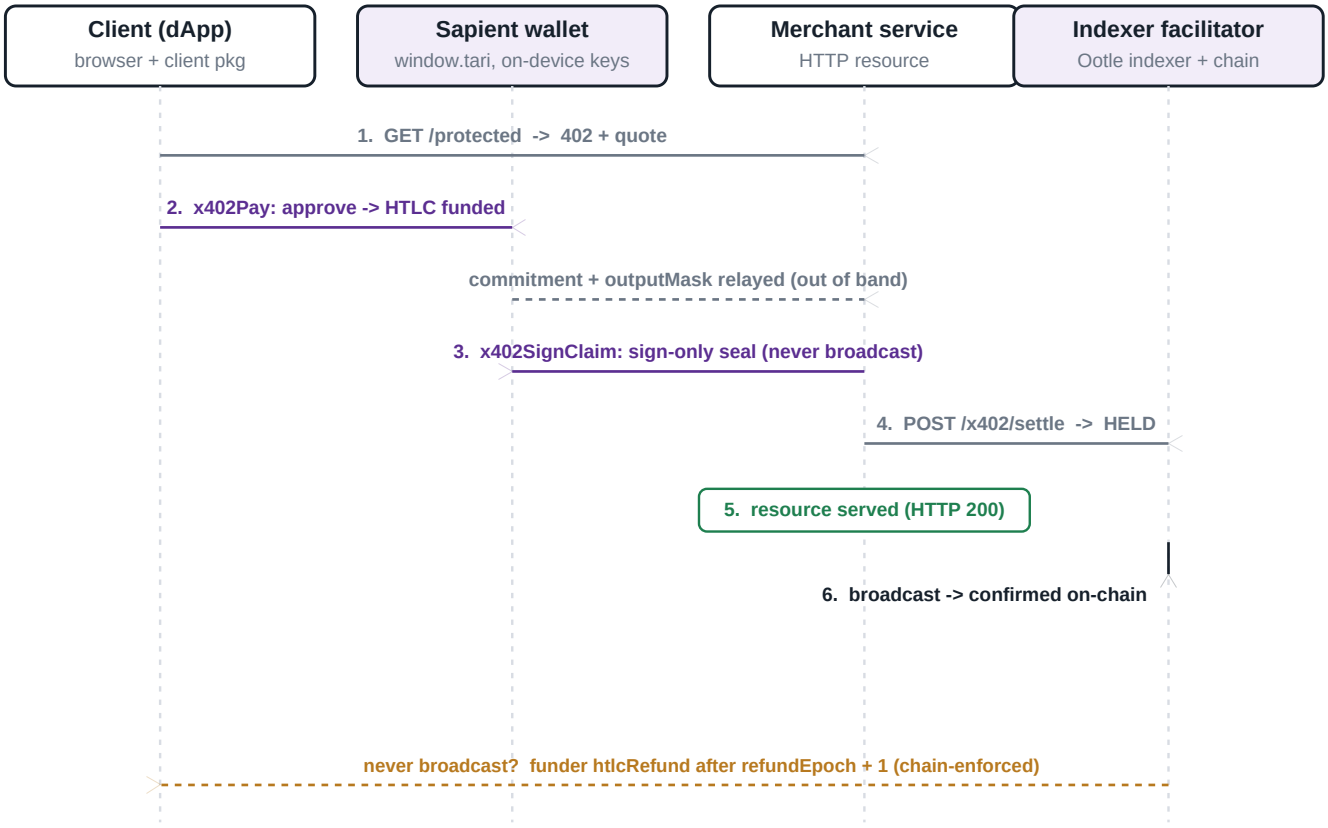
The fork (branch `x402/quote-and-signonly`) adds four coordinated pieces around Sapient's existing primitives:

Piece	What it does
Quote envelope (x402Pay)	A 402 quote becomes a first-class wallet request: the popup shows what the user is buying, the price, the fee, and the refund deadline in wall-clock time. Approving it funds an HTLC-locked stealth output (an htlcFund underneath) with a two-leaf condition tree: hash-lock claim OR time-locked refund.
Sign-only claim (x402SignClaim)	A new signing mode: build, sign and **seal** the HTLC claim transaction but do NOT broadcast it. The sealed payload goes back to the dApp with status signed-unbroadcast, so history never implies funds moved.
Facilitator spec (indexer as settlement)	docs/x402-facilitator.md pins the settlement interface: POST /x402/settle validates the sealed claim natively against the indexer's own chain index, HOLDS it, the merchant serves, then the indexer broadcasts. Idempotent, structured errors, no dry-run stage.

Piece	What it does
Client package (@potr3288/x402-ootle)	A zero-dependency TypeScript package: wraps fetch to parse the 402, drives the wallet requests, persists the refund mask, settles, and refunds on expiry.

4. The flow, end to end

The ordering is merchant-initiated hold-then-broadcast — chosen over broadcast-immediately (merchant owns funds before serving) and client-receipt-triggered (client gets a free-serve option). The hold window is what makes the HTLC meaningful:



Steps 1–4 happen with nothing on chain but the funding itself. The claim only reaches the chain after the merchant has confirmed settlement — and if the facilitator never broadcasts, the funder's refund path stays intact the entire time.

5. The refund backstop and honest trust semantics

The HTLC is not a delivery proof — HTTP delivery cannot be proven to a payment layer. What it guarantees is client-side abort plus automatic recovery: while the claim is held, the client's refund path is untouched, and once the refund epoch passes the funds return to the funder with nobody's cooperation. The indexer's hold power is disclosed rather than hidden: it can withhold broadcast (a merchant-side trust decision, auditable on chain), it cannot redirect or spend the funds (the claim is bound to payee, amount and hashlock, and the indexer holds no re-signing key), and a late broadcast is rejected on-chain by the claim's own epoch guard.

HTLC time-lock timeline (live-verified on esmeralda):

Run U failed: wallet dry-ran at epoch 12753 vs predicate AfterEpoch(12754) — the wallet's epoch view lags the probe across the boundary; AfterEpoch(N) is STRICT (spendable at N+1). The green band opens one epoch after refundEpoch — run V refunded there, at 12759 = 12758 + 1.

6. Verified live on esmeralda

Both scenarios ran against the public esmeralda indexer through the real extension UI (real popup approvals, deterministic wallets, a frozen facilitator for the refund case):

Scenario	Result	On-chain evidence
Happy path — pay, sign, hold, serve, broadcast	PASS	Claim tx 21257dd3cb0e…891a0b2 finalized with Commit; HTLC output confirmed spent.
Refund — facilitator frozen, claim never broadcast	PASS	Settlement reports expired; funder's htlcRefund tx 52a51a16ee86…ec9f1 finalized with Commit; HTLC output refunded.

Two product-level bugs surfaced and were fixed along the way: the SDK's HTLC verification was key-order sensitive (fixed by canonicalizing the relayed condition tree before verification), and the settlement design initially assumed a dry-run preflight the protocol cannot support (removed; broadcast itself is the admissibility proof). The refund leg also pinned down the chain's epoch semantics: AfterEpoch(N) is strict — spendable at N+1.

7. Where it goes next

The rail is scoped for upstream contribution: a scheme spec for the x402 foundation repo (which requires spec-first approval), an SDK implementation, and an e2e catalog entry — all drafted. The private-fee claim lane (keeping the merchant's address off the claim entirely) is the next wallet-side milestone; it needs a small SDK export of the envelope builder.